AGT — Architecture de Services Partagés

Cahier des Charges

Service Chatbot IA Centralisé

Le cerveau conversationnel multi-IA de l'écosystème

Dépendances validées	Auth v2.1, Users v1.0, Chat v1.2, Notification v1.2, Média v1.4
Version	1.2
Date	Mars 2026
Statut	Implementation-ready (validé architecture)
Classification	Confidentiel

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-03-20	Équipe Architecture	Création initiale du cahier des charges
1.1	2026-03-30	Équipe Architecture	Refonte multi-provider, abstraction canaux, pipeline ingestion, transfert humain, persistance analytique, branching conditionnel flows
1.2	2026-03-30	Équipe Architecture	Conversation Orchestrator formalisé, Action System standardisé, frontière Search Service documentée, statut Implementation-ready

Table des matières

1 Introduction **4**

2 Contexte & Problématique **4**
- 2.1 Contexte . . . 4
- 2.2 Problématique . . . 4

3 Vision & Objectifs **4**
- 3.1 Vision . . . 4
- 3.2 Objectifs . . . 5

4 Parties Prenantes & Utilisateurs **5**

5 Besoins Fonctionnels **5**
- 5.1 Gestion des Bots . . . 5
- 5.2 Architecture IA Multi-Couches . . . 5
  - 5.2.1 Couche 1 — Mots-clés / Intentions . . . 5
  - 5.2.2 Couche 2 — Flux Conversationnels . . . 6
  - 5.2.3 Couche 3 — IA Générative . . . 6
  - 5.2.4 Couche 4 — Fallback . . . 6
- 5.3 Base de Connaissances . . . 6
- 5.4 Mémoire de Conversation . . . 6
- 5.5 Actions & Intégrations . . . 6
- 5.6 Multi-langue . . . 7
- 5.7 Canaux / Connecteurs . . . 7
- 5.8 Transfert Humain . . . 7
- 5.9 Ingestion Documentaire . . . 7
- 5.10 Statistiques . . . 7

6 Besoins Non-Fonctionnels **8**

7 Architecture Technique **8**
- 7.1 Vue d'ensemble . . . 8
- 7.2 Architecture IA Multi-Couches — Pipeline de traitement . . . 8
- 7.3 Composant Central : Conversation Orchestrator . . . 8
  - 7.3.1 Responsabilités . . . 9
  - 7.3.2 Flux d'exécution . . . 9
- 7.4 Action System — Contrat Standardisé . . . 9

7.4.1 Contrat d'une Action . . . 9
7.4.2 Types d'actions supportés . . . 10
7.4.3 Règles d'exécution . . . 10
7.5 Frontière avec le Service Recherche . . . 10
7.5.1 Justification . . . 10
7.5.2 Point d'intégration futur . . . 10
7.6 Stack Technique . . . 11
7.7 Dépendances Externes & Services . . . 11
7.8 Workers Asynchrones (Celery) . . . 11
7.9 Stratégie de Persistance . . . 11

8 Modélisation **12**
8.1 Diagramme de Classes . . . 12
8.2 MLD — Modèle Logique de Données . . . 12
8.2.1 Table `bots` . . . 13
8.2.2 Table `bot_configs` . . . 13
8.2.3 Table `bot_channels` . . . 13
8.2.4 Table `knowledge_base_entries` . . . 14
8.2.5 Table `knowledge_categories` . . . 14
8.2.6 Table `knowledge_documents` . . . 14
8.2.7 Table `intents` . . . 14
8.2.8 Table `intent_keywords` . . . 14
8.2.9 Table `conversation_flows` . . . 15
8.2.10 Table `flow_nodes` . . . 15
8.2.11 Table `flow_actions` . . . 15
8.2.12 Table `bot_actions` . . . 15
8.2.13 Table `ai_provider_configs` . . . 16
8.2.14 Table `bot_stats` . . . 16
8.2.15 Table `conversation_logs` . . . 16
8.2.16 Table `transfer_logs` . . . 17
8.2.17 Table `ingestion_jobs` . . . 17

9 Contrats d'API **18**
9.1 Bots . . . 18
9.2 Knowledge Base . . . 19
9.3 Intentions . . . 20
9.4 Flux Conversationnels . . . 21
9.5 Conversation — Endpoint Principal . . . 22
9.6 Configuration IA (Multi-Provider) . . . 23
9.7 Statistiques . . . 25
9.8 Canaux / Connecteurs . . . 26
9.9 Transferts Humains . . . 27
9.10 Ingestion Documentaire . . . 28

10 Infrastructure **28**
10.1 Composants d'infrastructure . . . 29
10.2 Services externes à prévoir . . . 29

1 Introduction

Ce document constitue le cahier des charges du **Service Chatbot IA Centralisé** (ci-après « Chatbot Service ») destiné à l'écosystème de plateformes AGT. Il sert de référence unique pour la conception, le développement et l'intégration avec les autres services.

Périmètre

Le Chatbot Service est le **cerveau IA conversationnel multi-provider** de l'écosystème. Il gère l'ensemble du cycle de traitement des messages : détection d'intention, navigation dans des flux conversationnels, génération de réponses par IA, gestion de la mémoire de conversation et déclenchement d'actions métier. Il est conçu comme un service **générique, configurable et multi-canal**, sans logique métier codée en dur. Chaque plateforme configure ses propres bots.

Modes d'utilisation : Le service fonctionne (1) via le Hub Engine (AGT-Bot → WhatsApp), (2) en standalone (AGT-Market, SALMA, applications web), et (3) via connecteur externe (agent vocal, Messenger, Telegram, SMS, etc.).

Nouveautés v1.1 → v1.2 : Conversation Orchestrator formalisé comme composant central, Action System standardisé (contrat, retry, timeout, auth), frontière avec Search Service documentée (vector search interne justifié), statut *implementation-ready*.

2 Contexte & Problématique

2.1 Contexte

L'organisation développe plusieurs plateformes indépendantes nécessitant des capacités conversationnelles intelligentes :

Plateforme	Statut	Besoin IA
AGT-Bot	En développement	Chatbot WhatsApp multi-tenant via Hub Engine
AGT-Market	En développement	Assistant achat, support vendeur
SALMA	Migration prévue	Assistant conversationnel intégré
Futures	Planifié	Capacités IA à la demande

2.2 Problématique

Sans service centralisé, chaque plateforme implémenterait sa propre couche IA, entraînant : duplication des intégrations avec les providers IA (coûts & maintenance), incohérence de l'expérience conversationnelle, impossibilité de mutualiser les bases de connaissances, et multiplication des budgets IA sans contrôle.

3 Vision & Objectifs

3.1 Vision

Un service IA conversationnel unique, générique et configurable, permettant à toute plateforme de l'écosystème de déployer des chatbots intelligents en quelques minutes, sans développement spécifique.

3.2 Objectifs

#	Objectif	Indicateur de succès
O1	Service IA centralisé	Un seul point d'intégration pour toutes les plateformes
O2	Architecture multi-couches	4 couches de traitement (mots-clés, flux, IA générative, fallback)
O3	Bots génériques	Configuration no-code des bots, intentions, flux
O4	Multi-provider IA	Plusieurs providers par bot, routing par couche, fallback entre providers
O5	Multi-canal	Abstraction canal (WhatsApp, web, vocal, SMS, Messenger...) avec adapters
O6	Performance	$<$ 2s couches 1 & 2, $<$ 5s couche IA
O7	Multi-langue	FR + EN minimum, détection automatique
O8	Transfert humain	Escalade vers Service Chat avec contexte complet

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités IA
Équipe Backend	Interne	Développement du service Chatbot
Équipe Frontend	Interne	Intégration des widgets de chat
Équipe IA/ML	Interne	Configuration des modèles et prompts
DevOps	Interne	Infrastructure, monitoring, scaling
Administrateurs	Interne	Configuration des bots, intentions, flux
Utilisateurs finaux	Externe	Interaction avec les chatbots

5 Besoins Fonctionnels

5.1 Gestion des Bots

ID	Fonctionnalité	Priorité
F-BOT-01	Création d'un bot avec nom, avatar, personnalité, langues, prompt système	Urgent
F-BOT-02	Configuration des messages (accueil, fallback, transfert humain)	Urgent
F-BOT-03	Activation / désactivation d'un bot	Urgent
F-BOT-04	Duplication d'un bot existant	Normale
F-BOT-05	Définition du seuil de transfert humain (N échecs)	Urgent

5.2 Architecture IA Multi-Couches

Règle fondamentale : Chaque message entrant traverse séquentiellement 4 couches de traitement. La première couche renvoyant un résultat avec un score de confiance suffisant produit la réponse.

5.2.1 Couche 1 — Mots-clés / Intentions

ID	Fonctionnalité	Priorité
F-INT-01	CRUD des intentions avec mots-clés associés	Urgent
F-INT-02	Mapping mots-clés $\rightarrow$ intention avec score de confiance	Urgent
F-INT-03	Configuration du seuil de confiance par bot	Urgent
F-INT-04	Endpoint de test d'une intention	Normale

5.2.2 Couche 2 — Flux Conversationnels

ID	Fonctionnalité	Priorité
F-FLW-01	CRUD des flux conversationnels	Urgent
F-FLW-02	Arbre de décision avec nœuds : question, réponse, condition, action	Urgent
F-FLW-03	Configuration no-code friendly des flux	Urgent
F-FLW-04	Simulation / test d'un flux	Normale

5.2.3 Couche 3 — IA Générative

ID	Fonctionnalité	Priorité
F-GEN-01	Intégration multi-provider IA (OpenAI, Anthropic, Google Gemini, etc.)	Urgent
F-GEN-02	Contexte enrichi : base de connaissances + historique + personnalité	Urgent
F-GEN-03	Routing par couche : provider dédié par usage (RAG, conversation, traduction)	Urgent
F-GEN-04	Fallback entre providers (si provider A échoue → provider B)	Urgent
F-GEN-05	Configuration prompt système par bot	Urgent
F-GEN-06	Paramètres modèle (température, max tokens) configurables par provider	Normale
F-GEN-07	Circuit breaker par provider (désactivation auto si taux d'erreur élevé)	Normale
F-GEN-08	Health check périodique des providers configurés	Normale

5.2.4 Couche 4 — Fallback

ID	Fonctionnalité	Priorité
F-FBK-01	Message de fallback configurable par bot	Urgent
F-FBK-02	Compteur d'échecs consécutifs par conversation	Urgent
F-FBK-03	Logging des messages non résolus (persistance PostgreSQL)	Urgent

5.3 Base de Connaissances

ID	Fonctionnalité	Priorité
F-KB-01	CRUD des entrées (FAQ, texte libre, documents PDF)	Urgent
F-KB-02	Import CSV en masse	Normale
F-KB-03	Catégorisation des entrées	Normale
F-KB-04	Recherche par similarité (vector search)	Urgent
F-KB-05	Données dynamiques via API externe	Normale

5.4 Mémoire de Conversation

ID	Fonctionnalité	Priorité
F-MEM-01	Historique des N derniers messages par conversation	Urgent
F-MEM-02	Variables de session (contexte utilisateur)	Urgent
F-MEM-03	TTL configurable (ex : 30 min d'inactivité)	Urgent
F-MEM-04	Stockage Redis (pas de persistance long terme)	Urgent

5.5 Actions & Intégrations

Le bot peut déclencher des actions métier via des API internes ou des workflows configurés :

- Créer une commande (AGT-Market)
- Prendre un rendez-vous (SALMA)
- Envoyer un lien de paiement
- Transférer vers un agent humain (Service Chat)
- Appeler un webhook externe

5.6 Multi-langue

ID	Fonctionnalité	Priorité
F-LNG-01	Support FR + EN minimum	Urgent
F-LNG-02	Détection automatique de la langue	Urgent
F-LNG-03	Réponse dans la langue détectée	Urgent

5.7 Canaux / Connecteurs

ID	Fonctionnalité	Priorité
F-CHN-01	Enregistrement d'un canal par bot (WhatsApp, web, vocal, SMS, Messenger, API)	Urgent
F-CHN-02	Configuration par canal : format réponse, taille max, médias supportés, rate limit	Urgent
F-CHN-03	Adapter pattern : normalisation entrée/sortie par canal	Urgent
F-CHN-04	Webhook entrant configurable par canal (pour intégrations push)	Normale
F-CHN-05	Activation / désactivation d'un canal sans impacter le bot	Normale

5.8 Transfert Humain

ID	Fonctionnalité	Priorité
F-TRF-01	Déclenchement automatique après N fallbacks consécutifs	Urgent
F-TRF-02	Appel S2S vers Service Chat `POST /api/v1/chat/conversations/transfer`	Urgent
F-TRF-03	Transmission du contexte (bot_history, variables session, intent détecté)	Urgent
F-TRF-04	Suivi du statut transfert (en_attente, pris_en_charge, cloture)	Urgent
F-TRF-05	Reprise automatique par le bot après clôture opérateur	Normale
F-TRF-06	Déclenchement manuel via action dans un flow	Normale

5.9 Ingestion Documentaire

ID	Fonctionnalité	Priorité
F-ING-01	Upload de documents (PDF, TXT, DOCX, CSV) via Média Service	Urgent
F-ING-02	Worker asynchrone : extraction texte → chunking → embedding	Urgent
F-ING-03	Statut de traitement par document (pending, processing, done, error)	Urgent
F-ING-04	Ré-ingestion manuelle d'un document modifié	Normale
F-ING-05	Suppression en cascade des embeddings lors de suppression document	Normale

5.10 Statistiques

ID	Fonctionnalité	Priorité
F-STA-01	Volume de messages par bot	Urgent
F-STA-02	Taux de résolution (sans fallback)	Urgent
F-STA-03	Taux de fallback	Urgent
F-STA-04	Top intentions détectées	Normale
F-STA-05	Messages non répondus	Normale

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Temps de réponse couches 1 & 2	< 2s
NF-02	Performance	Temps de réponse couche IA	< 5s
NF-03	Performance	Throughput	200 req/s
NF-04	Disponibilité	Uptime	99.9%
NF-05	Sécurité	Authentification Bearer via Service Auth	Obligatoire
NF-06	Sécurité	Auth S2S pour appels inter-services (transfert, etc.)	Obligatoire
NF-07	Sécurité	Chiffrement des clés API IA (AES-256)	Obligatoire
NF-08	Sécurité	Rate limiting par bot et par canal	Configurable
NF-09	Scalabilité	Architecture stateless, multi-instance	Obligatoire
NF-10	Données	Mémoire conversation : Redis TTL (court terme)	Obligatoire
NF-11	Données	Logs analytiques : PostgreSQL (long terme)	Obligatoire
NF-12	Observabilité	Logs structurés JSON, request ID, correlation ID	Obligatoire
NF-13	Résilience	Fallback obligatoire si toutes couches échouent	Obligatoire
NF-14	Résilience	Circuit breaker par provider IA	Obligatoire
NF-15	Extensibilité	Multi-provider IA par bot (Strategy + routing)	Obligatoire
NF-16	Extensibilité	Multi-canal via adapter pattern	Obligatoire

7 Architecture Technique

7.1 Vue d'ensemble

Le Chatbot Service est un microservice REST stateless, déployé en conteneur Docker. Il expose une API JSON consommée par les plateformes clientes (via Hub Engine ou en direct) et interagit avec les providers IA externes.

7.2 Architecture IA Multi-Couches — Pipeline de traitement

Pipeline séquentiel : Message → **Couche 1** (mots-clés) → **Couche 2** (flux) → **Couche 3** (IA générative) → **Couche 4** (fallback). Chaque couche peut court-circuiter le pipeline si le score de confiance dépasse le seuil configuré.

7.3 Composant Central : Conversation Orchestrator

Le **Conversation Orchestrator** est le composant interne qui pilote le cycle de vie complet d'un message entrant. Il n'est pas un microservice séparé : c'est une classe/module Python central dans le codebase du Chatbot Service.

7.3.1 Responsabilités

Responsabilité	Détail
Channel Validation	Vérifie que le canal est actif pour le bot, charge la config canal (adapter)
Context Loading	Charge le contexte conversation depuis Redis (historique, variables, compteur fallbacks)
Language Detection	Détecte la langue du message si non fournie
Pipeline Routing	Exécute séquentiellement les couches 1 → 2 → 3 → 4, court-circuite dès confiance suffisante
Provider Selection	Pour la couche 3, sélectionne le provider selon `purpose` + `priority`, applique le circuit breaker
Action Execution	Si une couche ou un flow déclenche une action, exécute via l'Action Executor
Transfer Decision	Si `consecutive_fallbacks` ≥ `human_transfer_after`, déclenche le transfert S2S vers Chat
Response Adaptation	Adapte la réponse au canal (taille, format, médias) via le Channel Adapter
Context Saving	Sauvegarde le contexte mis à jour dans Redis
Logging	Écrit de manière asynchrone dans `conversation_logs` (fire-and-forget via Celery)

7.3.2 Flux d'exécution

1. `POST /chatbot/converse` reçu par le contrôleur Django
2. Le contrôleur instancie l'`Orchestrator` avec le payload
3. L'Orchestrator charge le bot, valide le canal, charge le contexte Redis
4. L'Orchestrator exécute le pipeline :
   (a) **Couche 1** — `IntentResolver` : matching mots-clés → si confiance ≥ seuil, retourne
   (b) **Couche 2** — `FlowEngine` : si un flow est actif ou déclenché par intent, exécute les nœuds (avec branching conditionnel)
   (c) **Couche 3** — `AiEngine` : sélectionne le provider (`purpose=conversation`), construit le prompt (system_prompt + KB context + historique), appelle l'API. Si échec, tente le fallback provider (`priority=1`)
   (d) **Couche 4** — `FallbackHandler` : retourne le `fallback_message`, incrémente le compteur, évalue le transfert humain
5. L'Orchestrator adapte la réponse via le `ChannelAdapter`
6. L'Orchestrator sauvegarde le contexte dans Redis, enqueue le log async
7. Le contrôleur retourne la réponse JSON

Règle : L'Orchestrator ne contient AUCUNE logique métier. Il orchestre des composants configurables. Toute la "connaissance" vient de la base de données (intentions, flows, prompts, actions).

7.4 Action System — Contrat Standardisé

Toute action déclenchée par un flow node ou par le bot (table `bot_actions`, `flow_actions`) suit un contrat standardisé exécuté par l'**Action Executor**, un sous-composant de l'Orchestrator.

7.4.1 Contrat d'une Action

```
{
  "type": "api_call",
  "service": "market",
  "endpoint": "/api/v1/orders",
  "method": "POST",
  "auth": "s2s",
  "headers": {"X-Bot-Id": "{{bot_id}}"},
  "payload_template": {
```

```
    "user_id": "{{sender_id}}",
    "product_id": "{{flow_var.product_id}}"
  },
  "timeout_ms": 3000,
  "retry": {
    "enabled": true,
    "max_attempts": 2,
    "backoff_ms": 500
  },
  "on_failure": "continue"
}
```

7.4.2 Types d'actions supportés

Type	Auth	Description
`api_call`	S2S / Bearer	Appel REST vers un service interne AGT
`webhook`	Custom headers	Appel HTTP vers un endpoint externe
`transfer_human`	S2S	Déclenche un transfert vers le Service Chat
`send_notification`	S2S	Envoie une notification via le Service Notification
`set_variable`	—	Modifie une variable de session (Redis)

7.4.3 Règles d'exécution

- **Auth S2S :** utilise le `client_id` / `client_secret` du Chatbot Service enregistré dans Auth v2.1 (endpoint `POST /auth/s2s/token`).
- **Template variables :** les `{{...}}` sont résolus au runtime depuis le contexte (sender_id, bot_id, flow variables, session variables).
- **Timeout :** défaut 3000ms, configurable par action.
- **Retry :** optionnel, exponential backoff, max 3 tentatives.
- **on_failure :** `continue` (ignore et poursuit le flow) ou `abort` (retourne une erreur au user).
- **Logging :** chaque exécution d'action est loggée (succès/échec, durée, HTTP status).

7.5 Frontière avec le Service Recherche

Décision d'architecture : Le Chatbot Service possède sa **propre** capacité de vector search (pgvector) pour la knowledge base interne des bots. Il n'utilise PAS le Service Recherche v1.2 pour cela.

7.5.1 Justification

Critère	Service Recherche v1.2	Chatbot KB (pgvector)
Périmètre	Données métier plateformes (produits, FAQ, formations)	Knowledge base privée par bot (RAG)
Technologie	Elasticsearch (full-text + futur dense_vector)	pgvector (similarité cosinus native PostgreSQL)
Latence cible	$<$ 200ms	$<$ 100ms (critique pour pipeline temps réel)
Cycle de vie	Index gérés par les plateformes	Embeddings gérés par le pipeline d'ingestion interne
Sémantique MVP	Full-text + scoring (embeddings = roadmap future)	Vector search = MVP day 1

7.5.2 Point d'intégration futur

Quand le Service Recherche v1.2+ implémentera la recherche sémantique par embeddings (dense_vector Elasticsearch), le Chatbot Service pourra optionnellement déléguer la recherche KB via un adapter :

— `search_backend: "internal"` → pgvector (défaut, MVP)
— `search_backend: "search_service"` → appel REST vers Search Service

Ce choix sera configurable par bot via `bot_configs` (`config_key: "search_backend"`).

7.6 Stack Technique

Composant	Technologie
Langage & Framework	Python 3.11+ / Django REST Framework
Base de données	PostgreSQL 15+
Cache / Contexte	Redis 7+ (mémoire conversation, sessions, rate limiting)
Recherche vectorielle	pgvector ou Elasticsearch
Providers IA	OpenAI API, Anthropic API (interchangeables)
Message Broker	RabbitMQ ou Redis Streams
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

7.7 Dépendances Externes & Services

Service	Type	Usage
Auth Service	Interne	Validation des tokens Bearer + auth S2S (obligatoire)
Chat Service	Interne	Transfert vers agent humain (S2S : `POST /api/v1/chat/conversations/transfer`)
User Service	Interne	Données utilisateur pour contexte, rôles admin
Notification Service	Interne	Notifications push liées aux conversations
Média Service	Interne	Gestion des fichiers / images / documents uploadés
OpenAI / Anthropic / Google	Externe	Providers IA générative (couche 3) — multi-provider par bot
Elasticsearch / pgvector	Interne	Recherche par similarité (knowledge base)

Règle d'architecture : Le Chatbot Service ne contient aucune logique métier en dur. Toute la configuration (intentions, flux, prompts, actions) est stockée en base et pilotée par API.

7.8 Workers Asynchrones (Celery)

Le service utilise des workers Celery pour les tâches lourdes ou périodiques :

Worker	Déclencheur	Rôle
`ingest_document`	Event (upload doc)	Extraction texte → chunking → embedding → insertion KB
`aggregate_stats`	Cron (toutes les heures)	Agrège `conversation_logs` → insère/met à jour `bot_stats`
`health_check_providers`	Cron (toutes les 5 min)	Ping les providers IA, met à jour circuit breaker
`cleanup_redis_sessions`	Cron (quotidien)	Purge les sessions expirées dans Redis

7.9 Stratégie de Persistance

Séparation mémoire / analytique :

— **Redis** (TTL configurable, défaut 30 min) : contexte conversation en cours, variables de session, compteur fallbacks consécutifs. Pas de persistance long terme.
— **PostgreSQL** `conversation_logs` : chaque échange est loggé de manière asynchrone (fire-and-forget) pour les analytics, le dashboard stats, et le dataset de fine-tuning.

— **PostgreSQL** `bot_stats` : agrégation horaire par le worker `aggregate_stats` à partir des `conversation_logs`.

8 Modélisation

8.1 Diagramme de Classes

La figure 1 présente le diagramme de classes complet du Service Chatbot IA. Il illustre les 13 entités principales et leurs relations.

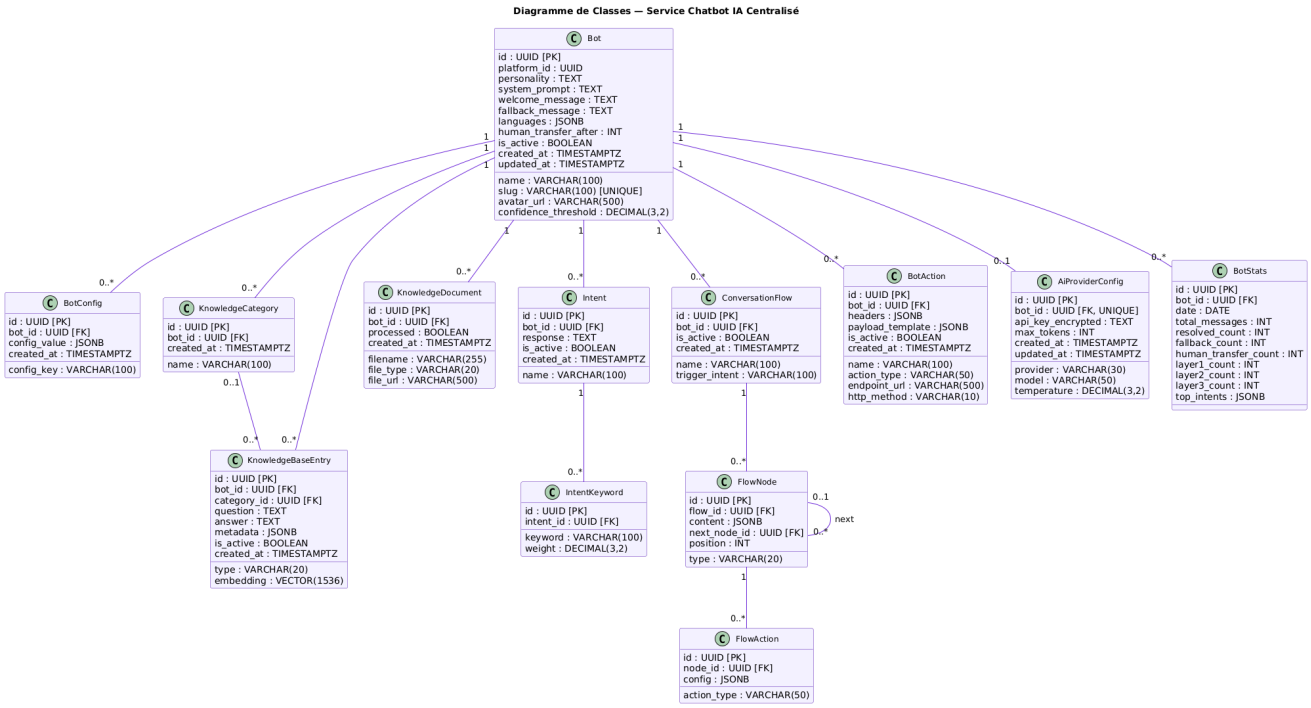

FIGURE 1 – Diagramme de classes — Service Chatbot IA Centralisé

8.2 MLD — Modèle Logique de Données

8.2.1 Table bots

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme propriétaire
name	VARCHAR(100)	NOT NULL	Nom du bot
slug	VARCHAR(100)	UNIQUE	Slug URL-friendly
avatar_url	VARCHAR(500)	NULL	URL de l'avatar
personality	TEXT	NULL	Description de la personnalité
system_prompt	TEXT	NULL	Prompt système pour l'IA
welcome_message	TEXT	NULL	Message d'accueil
fallback_message	TEXT	NULL	Message de fallback
languages	JSONB	DEFAULT '["fr"]'	Langues supportées
confidence_threshold	DECIMAL(3,2)	DEFAULT 0.7	Seuil de confiance
human_transfer_after	INT	DEFAULT 3	Transfert après N échecs
is_active	BOOLEAN	DEFAULT true	Bot actif
version	INT	DEFAULT 1	Version du bot (incrémentée à chaque publication)
status	VARCHAR(20)	DEFAULT 'draft'	draft, published, archived
created_at	TIMESTAMPTZ	NOT NULL	Date de création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière modification

8.2.2 Table bot_configs

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
config_key	VARCHAR(100)	NOT NULL	Clé de configuration
config_value	JSONB	NOT NULL	Valeur
created_at	TIMESTAMPTZ	NOT NULL	Création

Contrainte unique : (`bot_id`, `config_key`).

8.2.3 Table bot_channels

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
channel_type	VARCHAR(30)	NOT NULL	whatsapp, web, vocal, sms, messenger, api
display_name	VARCHAR(100)	NULL	Nom affiché
config	JSONB	NOT NULL	Config spécifique canal (max_length, media_types, format)
rate_limit	INT	DEFAULT 60	Requêtes max par minute par canal
is_active	BOOLEAN	DEFAULT true	Canal actif
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Modification

Contrainte unique : (`bot_id`, `channel_type`).

8.2.4 Table knowledge_base_entries

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
category_id	UUID	FK → knowledge_categories, NULL	Catégorie
type	VARCHAR(20)	NOT NULL	faq, document, text, api
question	TEXT	NULL	Question (si FAQ)
answer	TEXT	NOT NULL	Contenu / réponse
embedding	VECTOR(1536)	NULL	Vecteur pour recherche similarité
metadata	JSONB	NULL	Métadonnées additionnelles
is_active	BOOLEAN	DEFAULT true	Entrée active
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.5 Table knowledge_categories

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
name	VARCHAR(100)	NOT NULL	Nom de la catégorie
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.6 Table knowledge_documents

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
filename	VARCHAR(255)	NOT NULL	Nom du fichier
file_type	VARCHAR(20)	NOT NULL	pdf, txt, csv
file_url	VARCHAR(500)	NOT NULL	URL stockage (via Média Service)
status	VARCHAR(20)	DEFAULT 'pending'	pending, processing, done, error
chunk_count	INT	DEFAULT 0	Nombre de chunks extraits
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.7 Table intents

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
name	VARCHAR(100)	NOT NULL	Nom de l'intention
response	TEXT	NOT NULL	Réponse associée
is_active	BOOLEAN	DEFAULT true	Intention active
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.8 Table intent_keywords

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
intent_id	UUID	FK → intents	Intention associée
keyword	VARCHAR(100)	NOT NULL	Mot-clé
weight	DECIMAL(3,2)	DEFAULT 1.0	Poids du mot-clé

8.2.9 Table conversation_flows

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
name	VARCHAR(100)	NOT NULL	Nom du flux
trigger_intent	VARCHAR(100)	NULL	Intention déclencheuse
is_active	BOOLEAN	DEFAULT true	Flux actif
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.10 Table flow_nodes

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
flow_id	UUID	FK → conversation_flows	Flux parent
type	VARCHAR(20)	NOT NULL	question, response, condition, action
content	JSONB	NOT NULL	Contenu du nœud
next_node_id	UUID	FK → flow_nodes, NULL	Nœud suivant par défaut
branches	JSONB	NULL	Branching conditionnel : `["condition":"...", "next_node_id":"uuid"]`
position	INT	NOT NULL	Ordre dans le flux

8.2.11 Table flow_actions

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
node_id	UUID	FK → flow_nodes	Nœud associé
action_type	VARCHAR(50)	NOT NULL	api_call, webhook, transfer_human, send_notification, set_variable
config	JSONB	NOT NULL	Contrat Action standardisé (voir section Architecture → Action System)

8.2.12 Table bot_actions

Actions globales du bot (non liées à un flow spécifique). Utilisent le même contrat standardisé que les `flow_actions`.

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
name	VARCHAR(100)	NOT NULL	Nom de l'action
action_type	VARCHAR(50)	NOT NULL	api_call, webhook, transfer_human, send_notification, set_variable
config	JSONB	NOT NULL	Contrat Action standardisé (voir section Architecture → Action System)
is_active	BOOLEAN	DEFAULT true	Action active
created_at	TIMESTAMPTZ	NOT NULL	Création

8.2.13 Table ai_provider_configs

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
provider	VARCHAR(30)	NOT NULL	openai, anthropic, google, mistral, custom
model	VARCHAR(50)	NOT NULL	gpt-4o, claude-sonnet-4-20250514, gemini-pro, etc.
api_key_encrypted	TEXT	NOT NULL	Clé API chiffrée (AES-256)
base_url	VARCHAR(500)	NULL	URL custom (pour providers self-hosted)
temperature	DECIMAL(3,2)	DEFAULT 0.7	Température du modèle
max_tokens	INT	DEFAULT 1000	Tokens max par réponse
purpose	VARCHAR(30)	NOT NULL	conversation, rag, translation, fallback
priority	INT	DEFAULT 0	Priorité (0 = principal, 1+ = fallback)
is_active	BOOLEAN	DEFAULT true	Provider actif
circuit_breaker_threshold	INT	DEFAULT 5	Échecs consécutifs avant désactivation auto
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Modification

Contrainte unique : (`bot_id`, `provider`, `purpose`). Un bot peut avoir OpenAI pour conversation et Gemini pour RAG.

8.2.14 Table bot_stats

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
date	DATE	NOT NULL	Jour
total_messages	INT	DEFAULT 0	Nombre total de messages
resolved_count	INT	DEFAULT 0	Résolus sans fallback
fallback_count	INT	DEFAULT 0	Passés en fallback
human_transfer_count	INT	DEFAULT 0	Transférés vers humain
layer1_count	INT	DEFAULT 0	Résolus couche 1
layer2_count	INT	DEFAULT 0	Résolus couche 2
layer3_count	INT	DEFAULT 0	Résolus couche 3
top_intents	JSONB	NULL	Top intentions du jour

Contrainte unique : (`bot_id`, `date`).

8.2.15 Table conversation_logs

Persistance analytique des messages (séparée de la mémoire Redis).

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
conversation_id	UUID	NOT NULL	ID session (même que Redis)
sender_id	UUID	NOT NULL	Utilisateur
channel	VARCHAR(30)	NOT NULL	Canal d'origine
user_message	TEXT	NOT NULL	Message utilisateur
bot_response	TEXT	NOT NULL	Réponse du bot
layer_used	VARCHAR(30)	NOT NULL	layer_1_keywords, layer_2_flow, layer_3_ai, layer_4_fallback
intent_detected	VARCHAR(100)	NULL	Intention détectée
confidence	DECIMAL(3,2)	NULL	Score de confiance
provider_used	VARCHAR(30)	NULL	Provider IA utilisé (si couche 3)
tokens_used	INT	NULL	Tokens consommés
processing_time_ms	INT	NOT NULL	Temps de traitement
is_resolved	BOOLEAN	NOT NULL	Résolu sans fallback
created_at	TIMESTAMPTZ	NOT NULL	Horodatage

Index : (`bot_id`, `created_at` DESC), (`bot_id`, `is_resolved`).

8.2.16 Table `transfer_logs`

Suivi des transferts bot → humain côté Chatbot.

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
bot_id	UUID	FK → bots	Bot associé
conversation_id	UUID	NOT NULL	Session chatbot
chat_transfer_id	UUID	NULL	ID transfert côté Service Chat
user_id	UUID	NOT NULL	Utilisateur transféré
reason	VARCHAR(50)	NOT NULL	max_fallbacks, manual, flow_action
bot_history	JSONB	NOT NULL	Historique transmis au Chat
context	JSONB	NULL	Variables de session transmises
status	VARCHAR(20)	NOT NULL	pending, sent, accepted, closed, error
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ statut

8.2.17 Table `ingestion_jobs`

Pipeline d'ingestion documentaire asynchrone.

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
document_id	UUID	FK → knowledge_documents	Document source
status	VARCHAR(20)	NOT NULL	pending, processing, done, error
chunks_created	INT	DEFAULT 0	Nombre de chunks générés
error_message	TEXT	NULL	Détail erreur si applicable
started_at	TIMESTAMPTZ	NULL	Début traitement
completed_at	TIMESTAMPTZ	NULL	Fin traitement
created_at	TIMESTAMPTZ	NOT NULL	Création

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Tous les endpoints requièrent un header `Authorization: Bearer <token>` validé via le Service Auth.

9.1 Bots

POST /bots

Description : Création d'un nouveau bot.
Request body :

```
{
  "platform_id": "uuid",
  "name": "Assistant Market",
  "personality": "Friendly and helpful",
  "system_prompt": "Tu es un assistant...",
  "languages": ["fr", "en"],
  "welcome_message": "Bonjour ! Comment puis-je vous aider ?",
  "fallback_message": "D sol  , je n'ai pas compris.",
  "human_transfer_after": 3,
  "confidence_threshold": 0.7
}
```

Response 201 :

```
{
  "id": "uuid",
  "name": "Assistant Market",
  "slug": "assistant-market",
  "is_active": true,
  "created_at": "2026-03-20T10:00:00Z"
}
```

Codes : 201 Created, 400 Validation error, 401 Unauthorized.

GET /bots

Description : Liste des bots de la plateforme.
Query params : `?platform_id=uuid&page=1&limit=20`
Response 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "name": "Assistant Market",
      "is_active": true,
      "languages": ["fr", "en"],
      "created_at": "2026-03-20T10:00:00Z"
    }
  ],
  "total": 1,
  "page": 1
}
```

GET /bots/{id}

Description : Détail complet d'un bot.
Codes : 200 OK, 404 Not found.

`PUT /bots/{id}`

Description : Mise à jour d'un bot (mêmes champs que POST).
Codes : 200 OK, 400 Validation error, 404 Not found.

`DELETE /bots/{id}`

Description : Suppression d'un bot et de toutes ses données associées.
Codes : 200 OK, 404 Not found.

`POST /bots/{id}/duplicate`

Description : Duplication complète d'un bot (config, intentions, flux, knowledge base).
Request body :

```
{
  "new_name": "Assistant Market v2"
}
```

Response 201 : Retourne le nouveau bot créé.

`PUT /bots/{id}/status`

Description : Activation / désactivation d'un bot.
Request body :

```
{
  "is_active": false
}
```

Codes : 200 OK, 404 Not found.

9.2 Knowledge Base

`POST /bots/{id}/knowledge`

Description : Ajout d'une entrée dans la base de connaissances.
Request body :

```
{
  "type": "faq",
  "question": "Quels sont vos horaires ?",
  "answer": "Nous sommes ouverts de 8h a 18h.",
  "category_id": "uuid"
}
```

Codes : 201 Created, 400 Validation error.

`GET /bots/{id}/knowledge`

Description : Liste des entrées. Query params : `?type=faq&category_id=uuid&page=1&limit=20`
Codes : 200 OK.

`PUT /bots/{id}/knowledge/{entryId}`

Description : Mise à jour d'une entrée.
Codes : 200 OK, 404 Not found.

DELETE /bots/{id}/knowledge/{entryId}

Description : Suppression d'une entrée.
Codes : 200 OK, 404 Not found.

POST /bots/{id}/knowledge/import

Description : Import en masse depuis un fichier CSV.
Content-Type : `multipart/form-data`
Response 200 :

```
{
  "imported": 42,
  "errors": 2,
  "error_details": [
    {"line": 15, "reason": "Missing answer field"}
  ]
}
```

POST /bots/{id}/knowledge/search

Description : Recherche par similarité dans la base de connaissances.
Request body :

```
{
  "query": "Comment passer une commande ?",
  "limit": 5
}
```

Response 200 :

```
{
  "results": [
    {
      "id": "uuid",
      "question": "Comment commander ?",
      "answer": "Rendez-vous sur...",
      "score": 0.92
    }
  ]
}
```

9.3 Intentions

POST /bots/{id}/intents

Description : Création d'une intention avec ses mots-clés.
Request body :

```
{
  "name": "greeting",
  "response": "Bonjour ! Comment puis-je vous aider ?",
  "keywords": [
    {"keyword": "bonjour", "weight": 1.0},
    {"keyword": "salut", "weight": 0.9},
    {"keyword": "hello", "weight": 0.8}
  ]
}
```

Codes : 201 Created, 400 Validation error.

`GET /bots/{id}/intents`

Description : Liste des intentions du bot.
Codes : 200 OK.

`PUT /bots/{id}/intents/{intentId}`

Description : Mise à jour d'une intention et de ses mots-clés.
Codes : 200 OK, 404 Not found.

`DELETE /bots/{id}/intents/{intentId}`

Description : Suppression d'une intention.
Codes : 200 OK, 404 Not found.

`POST /bots/{id}/intents/test`

Description : Test de détection d'intention sur un message.
Request body :

```
{
  "message": "bonjour, je cherche de l'aide"
}
```

Response 200 :

```
{
  "detected_intent": "greeting",
  "confidence": 0.85,
  "all_matches": [
    {"intent": "greeting", "confidence": 0.85},
    {"intent": "help", "confidence": 0.60}
  ]
}
```

9.4 Flux Conversationnels

`POST /bots/{id}/flows`

Description : Création d'un flux conversationnel avec ses nœuds.
Request body :

```
{
  "name": "Flux commande",
  "trigger_intent": "order",
  "nodes": [
    {
      "type": "question",
      "content": {"text": "Quel produit souhaitez-vous ?"},
      "position": 1
    },
    {
      "type": "action",
      "content": {"action_type": "api_call", "endpoint": "/orders"},
      "position": 2
    }
  ]
```

```
}
```

Codes : 201 Created, 400 Validation error.

`GET /bots/{id}/flows`

Description : Liste des flux du bot.
Codes : 200 OK.

`GET /bots/{id}/flows/{flowId}`

Description : Détail d'un flux avec tous ses nœuds et actions.
Codes : 200 OK, 404 Not found.

`PUT /bots/{id}/flows/{flowId}`

Description : Mise à jour d'un flux.
Codes : 200 OK, 404 Not found.

`DELETE /bots/{id}/flows/{flowId}`

Description : Suppression d'un flux.
Codes : 200 OK, 404 Not found.

`POST /bots/{id}/flows/{flowId}/test`

Description : Simulation d'un flux avec un message de test.
Request body :

```
{
  "message": "je veux commander",
  "session_vars": {"user_name": "Alice"}
}
```

Response 200 :

```
{
  "flow_name": "Flux commande",
  "current_node": {"type": "question", "position": 1},
  "response": "Quel produit souhaitez-vous ?",
  "completed": false
}
```

9.5 Conversation — Endpoint Principal

`POST /chatbot/converse`

Description : Endpoint principal. Envoie un message et reçoit la réponse du bot après traitement par le pipeline multi-couches. Le champ `channel` est obligatoire et détermine l'adapter de sortie.

Request body :

```
{
  "bot_id": "uuid",
  "conversation_id": "uuid-or-null",
  "sender_id": "uuid",
  "channel": "whatsapp",
  "message": "Bonjour, je cherche un produit",
  "language": "fr",
  "metadata": {
```

```
    "platform": "agt-bot",
    "source_message_id": "wa-msg-123"
  }
}
```

Response 200 :

```
{
  "response": "Bonjour ! Quel type de produit recherchez-vous ?",
  "conversation_id": "uuid",
  "intent_detected": "greeting",
  "layer_used": "layer_1_keywords",
  "confidence": 0.88,
  "provider_used": null,
  "action": null,
  "language": "fr",
  "transfer": null,
  "metadata": {
    "processing_time_ms": 145,
    "fallback": false,
    "consecutive_fallbacks": 0
  }
}
```

Valeurs possibles de `layer_used` **:**

— `layer_1_keywords` — Résolu par mots-clés / intentions
— `layer_2_flow` — Résolu par flux conversationnel
— `layer_3_ai` — Résolu par IA générative
— `layer_4_fallback` — Fallback (aucune couche n'a résolu)

Si transfert humain déclenché, le champ `transfer` contient :

```
{
  "transfer": {
    "id": "uuid",
    "status": "sent",
    "reason": "max_fallbacks",
    "chat_transfer_id": "uuid"
  }
}
```

Codes : 200 OK, 400 Validation error, 401 Unauthorized, 404 Bot not found, 429 Rate limited.

`DELETE /chatbot/conversations/{conversationId}`

Description : Suppression du contexte d'une conversation (mémoire Redis).
Codes : 200 OK, 404 Not found.

9.6 Configuration IA (Multi-Provider)

`POST /bots/{id}/ai-providers`

Description : Ajout d'un provider IA pour un bot. Un bot peut avoir plusieurs providers avec des rôles différents (conversation, RAG, traduction, fallback).
Request body :

```
{
  "provider": "anthropic",
  "model": "claude-sonnet-4-20250514",
```

```
  "api_key": "sk-ant-...",
  "purpose": "conversation",
  "priority": 0,
  "temperature": 0.5,
  "max_tokens": 1500
}
```

Response 201 :

```
{
  "id": "uuid",
  "provider": "anthropic",
  "model": "claude-sonnet-4-20250514",
  "purpose": "conversation",
  "priority": 0,
  "is_active": true,
  "has_api_key": true
}
```

Codes : 201 Created, 400 Validation error, 409 Conflit (provider+purpose déjà configuré).

GET /bots/{id}/ai-providers

Description : Liste de tous les providers IA configurés pour un bot.
Response 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "provider": "anthropic",
      "model": "claude-sonnet-4-20250514",
      "purpose": "conversation",
      "priority": 0,
      "is_active": true,
      "has_api_key": true
    },
    {
      "id": "uuid",
      "provider": "google",
      "model": "gemini-pro",
      "purpose": "rag",
      "priority": 0,
      "is_active": true,
      "has_api_key": true
    }
  ]
}
```

Note : Les clés API ne sont jamais retournées en clair.

PUT /bots/{id}/ai-providers/{providerId}

Description : Mise à jour d'un provider IA (modèle, température, priorité, etc.).
Codes : 200 OK, 400 Validation error, 404 Not found.

`DELETE /bots/{id}/ai-providers/{providerId}`

Description : Suppression d'un provider IA.
Codes : 200 OK, 404 Not found.

`POST /bots/{id}/ai-providers/{providerId}/test`

Description : Test d'un provider spécifique avec un message.
Request body :

```
{
  "message": "Quelle est votre politique de retour ?"
}
```

Response 200 :

```
{
  "response": "Notre politique de retour permet...",
  "provider": "anthropic",
  "model": "claude-sonnet-4-20250514",
  "tokens_used": 234,
  "processing_time_ms": 1820
}
```

`GET /bots/{id}/ai-providers/health`

Description : Health check de tous les providers IA configurés pour un bot.
Response 200 :

```
{
  "providers": [
    {
      "id": "uuid",
      "provider": "anthropic",
      "purpose": "conversation",
      "status": "healthy",
      "last_check": "2026-03-30T10:00:00Z",
      "consecutive_errors": 0
    },
    {
      "id": "uuid",
      "provider": "google",
      "purpose": "rag",
      "status": "degraded",
      "last_check": "2026-03-30T10:00:00Z",
      "consecutive_errors": 3
    }
  ]
}
```

9.7 Statistiques

`GET /bots/{id}/stats`

Description : Statistiques agrégées d'un bot.
Query params : `?from=2026-03-01&to=2026-03-20`
Response 200 :

```
{
  "bot_id": "uuid",
  "period": {"from": "2026-03-01", "to": "2026-03-20"},
  "total_messages": 12450,
  "resolution_rate": 0.87,
  "fallback_rate": 0.08,
  "human_transfer_rate": 0.05,
  "by_layer": {
    "layer_1": 5200,
    "layer_2": 3800,
    "layer_3": 2400,
    "layer_4": 1050
  },
  "top_intents": [
    {"intent": "greeting", "count": 3200},
    {"intent": "order_status", "count": 1800}
  ]
}
```

GET /bots/{id}/stats/unanswered

Description : Messages non résolus (passés en fallback).
Query params : ?page=1&limit=20
Response 200 :

```
{
  "data": [
    {
      "message": "Je veux annuler ma commande 12345",
      "count": 15,
      "last_seen": "2026-03-20T14:30:00Z"
    }
  ],
  "total": 42
}
```

9.8 Canaux / Connecteurs

POST /bots/{id}/channels

Description : Enregistrement d'un canal pour un bot.
Request body :

```
{
  "channel_type": "whatsapp",
  "display_name": "WhatsApp AGT-Bot",
  "config": {
    "max_response_length": 4096,
    "supported_media": ["image", "document"],
    "response_format": "text"
  },
  "rate_limit": 120
}
```

Codes : 201 Created, 400 Validation error, 409 Canal déjà configuré pour ce bot.

`GET /bots/{id}/channels`

Description : Liste des canaux configurés pour un bot.
Codes : 200 OK.

`PUT /bots/{id}/channels/{channelId}`

Description : Mise à jour de la configuration d'un canal.
Codes : 200 OK, 404 Not found.

`DELETE /bots/{id}/channels/{channelId}`

Description : Suppression d'un canal.
Codes : 200 OK, 404 Not found.

`PUT /bots/{id}/channels/{channelId}/status`

Description : Activation / désactivation d'un canal.
Request body :

```
{
  "is_active": false
}
```

Codes : 200 OK, 404 Not found.

9.9 Transferts Humains

`GET /bots/{id}/transfers`

Description : Liste des transferts bot → humain pour un bot.
Query params : `?status=pending&page=1&limit=20`
Response 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "conversation_id": "uuid",
      "user_id": "uuid",
      "reason": "max_fallbacks",
      "status": "sent",
      "chat_transfer_id": "uuid",
      "created_at": "2026-03-30T10:00:00Z"
    }
  ],
  "total": 5,
  "page": 1
}
```

`POST /chatbot/transfers/{transferId}/callback`

Description : Callback S2S appelé par le Service Chat lorsque le statut d'un transfert change (pris_en_charge, cloture). Permet au chatbot de reprendre la conversation après clôture.
Auth : S2S uniquement.
Request body :

```
{
  "status": "cloture",
```

```
  "operator_id": "uuid",
  "closed_at": "2026-03-30T11:00:00Z"
}
```

Codes : 200 OK, 404 Not found.

9.10 Ingestion Documentaire

`POST /bots/{id}/knowledge/documents`

Description : Upload d'un document pour ingestion dans la knowledge base. Le fichier est stocké via le Média Service, puis un job asynchrone extrait, découpe et génère les embeddings.
Content-Type : `multipart/form-data`
Response 202 Accepted :

```
{
  "document_id": "uuid",
  "filename": "politique-retour.pdf",
  "status": "pending",
  "job_id": "uuid"
}
```

Codes : 202 Accepted, 400 Invalid file type, 413 File too large.

`GET /bots/{id}/knowledge/documents/{docId}/status`

Description : Statut d'ingestion d'un document.
Response 200 :

```
{
  "document_id": "uuid",
  "status": "done",
  "chunks_created": 24,
  "started_at": "2026-03-30T10:01:00Z",
  "completed_at": "2026-03-30T10:01:45Z"
}
```

`POST /bots/{id}/knowledge/documents/{docId}/reprocess`

Description : Relance l'ingestion d'un document (supprime les anciens chunks et ré-extrait).
Codes : 202 Accepted, 404 Not found.

10 Infrastructure

10.1 Composants d'infrastructure

Service	Technologie	Rôle
Backend	Python / Django REST	API REST du service Chatbot
Worker Celery	Python / Celery	Ingestion docs, agrégation stats, health check providers
Base de données	PostgreSQL 15+	Stockage bots, intentions, flux, KB, stats, logs analytiques
Cache / Mémoire	Redis 7+	Contexte conversation, sessions, rate limiting, circuit breakers
Recherche vectorielle	pgvector / Elasticsearch	Recherche par similarité (knowledge base)
Providers IA	OpenAI, Anthropic, Google Gemini	IA générative (couche 3) — multi-provider par bot
Message Broker	RabbitMQ / Redis Streams	Jobs asynchrones (ingestion, stats) + events inter-services
Conteneur	Docker + Docker Compose	Déploiement isolé et reproductible
Reverse Proxy	Nginx / Traefik	TLS, load balancing

10.2 Services externes à prévoir

Catégorie	Fournisseurs	Facturation
Providers IA	OpenAI, Anthropic, Google Gemini, Mistral	Pay-per-token (coût principal du service)
Recherche vectorielle	Elasticsearch Cloud / pgvector	Hébergement ou intégré à PostgreSQL
Hébergement	AWS, GCP, DigitalOcean	Conteneurs, PostgreSQL, Redis managés

Annexes — Cas d'Usage

Cas 1 : WhatsApp via Hub Engine

1. L'utilisateur envoie un message WhatsApp.
2. AGT-Bot (Hub Engine) reçoit le message et appelle `POST /chatbot/converse` avec `channel: "whatsapp"`.
3. Le pipeline vérifie que le canal `whatsapp` est actif pour ce bot, applique l'adapter de sortie (limite taille, format texte).
4. Le pipeline multi-couches traite le message.
5. La réponse est retournée au Hub Engine qui l'envoie via WhatsApp.

Cas 2 : Flux de commande (AGT-Market)

1. L'utilisateur dit « je veux commander » via le widget web (`channel: "web"`).
2. Couche 1 détecte l'intention `order`.
3. Couche 2 active le flux « Flux commande ».
4. Le bot guide l'utilisateur à travers les nœuds (produit, quantité, confirmation).
5. Un nœud `condition` vérifie la disponibilité via `branches` (si oui → confirmation, si non → suggestion).
6. Un nœud `action` appelle l'API de création de commande.

Cas 3 : Fallback puis transfert humain

1. Le bot ne comprend pas le message (couches 1, 2, 3 échouent).
2. Couche 4 envoie le message de fallback. Le compteur `consecutive_fallbacks` s'incrémente.
3. Le message est loggé dans `conversation_logs` (persistance PostgreSQL).

4. Après 3 échecs consécutifs (`human_transfer_after`), le bot appelle le Service Chat en S2S : `POST /api/v1/chat/conversations/transfer` avec `bot_history` et `context`.
5. Un `transfer_log` est créé côté chatbot avec le statut `sent`.
6. Quand l'opérateur clôture, le Service Chat appelle `POST /chatbot/transfers/{id}/callback` avec `status: "cloture"`.
7. Le bot reprend la conversation.

Cas 4 : Multi-langue

1. L'utilisateur envoie un message en anglais.
2. Le service détecte la langue (EN).
3. Le pipeline utilise les intentions et prompts adaptés à la langue.
4. La réponse est générée en anglais (le provider de `purpose: "conversation"` est utilisé).

Cas 5 : Agent vocal externe

1. Un service d'agent vocal externe appelle `POST /chatbot/converse` avec `channel: "vocal"` et un Bearer token valide.
2. L'adapter vocal formate la réponse (texte court, sans markdown, compatible TTS).
3. La réponse est retournée au service vocal qui la synthétise en audio.

Cas 6 : Ingestion d'un document PDF

1. L'admin uploade un PDF via `POST /bots/{id}/knowledge/documents`.
2. Le fichier est stocké via le Média Service.
3. Un job Celery est créé (`ingestion_jobs`, statut `pending`).
4. Le worker extrait le texte, découpe en chunks, génère les embeddings via le provider de `purpose: "rag"`.
5. Les chunks sont insérés dans `knowledge_base_entries` avec leurs embeddings.
6. Le statut du job passe à `done`.

Cas 7 : Failover multi-provider

1. Un message arrive en couche 3 (IA générative).
2. Le provider principal (`priority: 0`, `purpose: "conversation"`) est appelé.
3. L'appel échoue (timeout, erreur 500). Le circuit breaker incrémente le compteur.
4. Le service bascule automatiquement sur le provider de `priority: 1` (fallback).
5. Si le provider principal atteint `circuit_breaker_threshold` erreurs consécutives, il est temporairement désactivé.

Code Source PlantUML — Diagramme de Classes

Le code ci-dessous permet de régénérer le diagramme de classes (figure 1) via tout outil compatible PlantUML. **v1.1** : ajout de `BotChannel`, `ConversationLog`, `TransferLog`, `IngestionJob`, refonte `AiProviderConfig` (multi-provider), ajout `branches` à `FlowNode`, ajout `version/status` à `Bot`.

```
@startuml
skinparam classAttributeIconSize 0
skinparam classFontSize 11
skinparam classFontColor #000000
skinparam classHeaderBackgroundColor #7C3AED
skinparam classHeaderFontColor white
skinparam classBorderColor #5B21B6
skinparam classBackgroundColor #F3F0FF
skinparam arrowColor #6D28D9
title Diagramme de Classes -- Service Chatbot IA Centralise v1.2

class Bot {
  id : UUID [PK]
```

```
  platform_id : UUID
  name : VARCHAR(100)
  slug : VARCHAR(100) [UQ]
  avatar_url : VARCHAR(500)
  personality : TEXT
  system_prompt : TEXT
  welcome_message : TEXT
  fallback_message : TEXT
  languages : JSONB
  confidence_threshold : DECIMAL(3,2)
  human_transfer_after : INT
  is_active : BOOLEAN
  version : INT
  status : VARCHAR(20)
  created_at : TIMESTAMPTZ
  updated_at : TIMESTAMPTZ
}

class BotConfig {
  id : UUID [PK]
  bot_id : UUID [FK]
  config_key : VARCHAR(100)
  config_value : JSONB
  created_at : TIMESTAMPTZ
}

class BotChannel {
  id : UUID [PK]
  bot_id : UUID [FK]
  channel_type : VARCHAR(30)
  display_name : VARCHAR(100)
  config : JSONB
  rate_limit : INT
  is_active : BOOLEAN
  created_at : TIMESTAMPTZ
  updated_at : TIMESTAMPTZ
}

class KnowledgeCategory {
  id : UUID [PK]
  bot_id : UUID [FK]
  name : VARCHAR(100)
  created_at : TIMESTAMPTZ
}

class KnowledgeBaseEntry {
  id : UUID [PK]
  bot_id : UUID [FK]
  category_id : UUID [FK]
  type : VARCHAR(20)
  question : TEXT
  answer : TEXT
  embedding : VECTOR(1536)
  metadata : JSONB
  is_active : BOOLEAN
  created_at : TIMESTAMPTZ
}

class KnowledgeDocument {
  id : UUID [PK]
  bot_id : UUID [FK]
  filename : VARCHAR(255)
  file_type : VARCHAR(20)
  file_url : VARCHAR(500)
  status : VARCHAR(20)
  chunk_count : INT
  created_at : TIMESTAMPTZ
}

class IngestionJob {
  id : UUID [PK]
  document_id : UUID [FK]
  status : VARCHAR(20)
  chunks_created : INT
  error_message : TEXT
```

```
  started_at : TIMESTAMPTZ
  completed_at : TIMESTAMPTZ
  created_at : TIMESTAMPTZ
}

class Intent {
  id : UUID [PK]
  bot_id : UUID [FK]
  name : VARCHAR(100)
  response : TEXT
  is_active : BOOLEAN
  created_at : TIMESTAMPTZ
}

class IntentKeyword {
  id : UUID [PK]
  intent_id : UUID [FK]
  keyword : VARCHAR(100)
  weight : DECIMAL(3,2)
}

class ConversationFlow {
  id : UUID [PK]
  bot_id : UUID [FK]
  name : VARCHAR(100)
  trigger_intent : VARCHAR(100)
  is_active : BOOLEAN
  created_at : TIMESTAMPTZ
}

class FlowNode {
  id : UUID [PK]
  flow_id : UUID [FK]
  type : VARCHAR(20)
  content : JSONB
  next_node_id : UUID [FK]
  branches : JSONB
  position : INT
}

class FlowAction {
  id : UUID [PK]
  node_id : UUID [FK]
  action_type : VARCHAR(50)
  config : JSONB
}

class BotAction {
  id : UUID [PK]
  bot_id : UUID [FK]
  name : VARCHAR(100)
  action_type : VARCHAR(50)
  config : JSONB
  is_active : BOOLEAN
  created_at : TIMESTAMPTZ
}

class AiProviderConfig {
  id : UUID [PK]
  bot_id : UUID [FK]
  provider : VARCHAR(30)
  model : VARCHAR(50)
  api_key_encrypted : TEXT
  base_url : VARCHAR(500)
  temperature : DECIMAL(3,2)
  max_tokens : INT
  purpose : VARCHAR(30)
  priority : INT
  is_active : BOOLEAN
  circuit_breaker_threshold : INT
  created_at : TIMESTAMPTZ
  updated_at : TIMESTAMPTZ
}

class BotStats {
```

```
  id : UUID [PK]
  bot_id : UUID [FK]
  date : DATE
  total_messages : INT
  resolved_count : INT
  fallback_count : INT
  human_transfer_count : INT
  layer1_count : INT
  layer2_count : INT
  layer3_count : INT
  top_intents : JSONB
}

class ConversationLog {
  id : UUID [PK]
  bot_id : UUID [FK]
  conversation_id : UUID
  sender_id : UUID
  channel : VARCHAR(30)
  user_message : TEXT
  bot_response : TEXT
  layer_used : VARCHAR(30)
  intent_detected : VARCHAR(100)
  confidence : DECIMAL(3,2)
  provider_used : VARCHAR(30)
  tokens_used : INT
  processing_time_ms : INT
  is_resolved : BOOLEAN
  created_at : TIMESTAMPTZ
}

class TransferLog {
  id : UUID [PK]
  bot_id : UUID [FK]
  conversation_id : UUID
  chat_transfer_id : UUID
  user_id : UUID
  reason : VARCHAR(50)
  bot_history : JSONB
  context : JSONB
  status : VARCHAR(20)
  created_at : TIMESTAMPTZ
  updated_at : TIMESTAMPTZ
}

' === Relations ===
Bot "1" -- "0..*" BotConfig
Bot "1" -- "0..*" BotChannel
Bot "1" -- "0..*" KnowledgeCategory
Bot "1" -- "0..*" KnowledgeBaseEntry
Bot "1" -- "0..*" KnowledgeDocument
Bot "1" -- "0..*" Intent
Bot "1" -- "0..*" ConversationFlow
Bot "1" -- "0..*" BotAction
Bot "1" -- "0..*" AiProviderConfig
Bot "1" -- "0..*" BotStats
Bot "1" -- "0..*" ConversationLog
Bot "1" -- "0..*" TransferLog
KnowledgeCategory "0..1" -- "0..*" KnowledgeBaseEntry
KnowledgeDocument "1" -- "0..*" IngestionJob
Intent "1" -- "0..*" IntentKeyword
ConversationFlow "1" -- "0..*" FlowNode
FlowNode "1" -- "0..*" FlowAction
FlowNode "0..1" -- "0..*" FlowNode : next
@enduml
```

Conclusion Générale

Le Service Chatbot IA Centralisé v1.2 est le pilier conversationnel stratégique de l'écosystème AGT. Son architecture repose sur un **Conversation Orchestrator** explicite qui pilote un pipeline 4 couches, orchestre des providers IA multiples avec failover, adapte les réponses par canal, et exécute des actions

standardisées vers les services internes.

Les décisions structurantes de cette version :

- **Multi-provider** avec routing par `purpose` et circuit breaker — aucune dépendance à un fournisseur unique.
- **Multi-canal** via adapter pattern — WhatsApp, web, vocal, SMS, API extensible sans toucher au pipeline.
- **Action System standardisé** — contrat unique (auth, retry, timeout, template variables) pour toutes les interactions inter-services.
- **Persistance duale** — Redis pour la mémoire temps réel, PostgreSQL pour les analytics et le fine-tuning.
- **Frontière Search Service** documentée — pgvector interne pour le RAG MVP, adapter configurable vers Search Service quand celui-ci supportera les embeddings.
- **Transfert humain bidirectionnel** — intégration S2S complète avec le Service Chat v1.2.

Prochaines étapes : validation équipe, initialisation des développements par priorité, déploiement progressif Hub Engine (AGT-Bot) puis extension aux autres canaux.

Annexes — Bonus / Stratégies Avancées

Suggestion automatique des questions non répondues

Le service collecte les messages passés en fallback et les agrège par similarité. L'endpoint `GET /bots/{id}/stats/una` permet aux administrateurs de transformer ces messages en nouvelles intentions ou entrées de knowledge base.

Gestion du budget IA par bot

Chaque bot peut avoir un budget mensuel configurable (en tokens ou en montant). Le service tracked la consommation et peut désactiver la couche 3 si le budget est épuisé, basculant directement vers le fallback.

A/B Testing des réponses

Possibilité de configurer plusieurs réponses pour une même intention avec distribution pondérée, permettant de mesurer laquelle génère le meilleur engagement.

Stratégie de fine-tuning

Les conversations réussies (haute confiance, pas de fallback) peuvent être exportées pour constituer un dataset de fine-tuning, améliorant progressivement les performances de la couche IA.

Glossaire

Terme	Définition
Intent (Intention)	Action ou sujet détecté dans le message de l'utilisateur via des mots-clés.
Flow (Flux)	Arbre de décision guidant une conversation multi-étapes.
Knowledge Base	Base de connaissances du bot (FAQ, documents, textes).
Provider IA	Service externe fournissant les capacités d'IA générative (OpenAI, Anthropic, Google).
Strategy Pattern	Patron de conception permettant de changer d'algorithme (provider IA) à l'exécution.
Circuit Breaker	Pattern de résilience désactivant temporairement un provider défaillant.
Channel Adapter	Composant normalisant les entrées/sorties selon le canal (WhatsApp, web, vocal).
Embedding	Représentation vectorielle d'un texte pour la recherche par similarité.
Chunking	Découpage d'un document en segments pour l'indexation vectorielle.
Ingestion	Pipeline asynchrone : extraction → chunking → embedding d'un document.
Fallback	Réponse par défaut quand aucune couche ne parvient à résoudre le message.
TTL	Time To Live — durée de vie d'une donnée en cache (Redis).
Hub Engine	Composant AGT-Bot gérant la connexion WhatsApp et routant les messages.
Vector Search	Recherche basée sur la distance entre vecteurs (similarité sémantique).
S2S (Service-to-Service)	Authentification inter-services via client_id/client_secret (Auth v2.1).
Celery	Framework Python pour l'exécution de tâches asynchrones (workers).
Conversation Orchestrator	Composant central qui pilote le cycle de vie d'un message (routing pipeline, contexte, actions, adaptation canal).
Action Executor	Sous-composant de l'Orchestrator exécutant les actions standardisées (API calls, webhooks, transferts).
Channel Adapter	Composant normalisant les entrées/sorties selon les contraintes du canal (taille, format, médias).
Circuit Breaker	Pattern de résilience désactivant temporairement un provider IA défaillant après N erreurs consécutives.

Fin du document — AGT Engineering — v1.2